GUÍA DE ACTIVIDAD PRÁCTICA

Diseño y Patrones Arquitectónicos · Unidad 2 · Bosquejo de la versión 1 del reto con un patrón clásico

Programa	Facultad de Ingeniería · Posgrado	Asignatura	Diseño y Patrones Arquitectónicos
Unidad	Unidad 2 · Práctica opcional	Unidad / Corte	2 · Corte 1
Modalidad	Individual	Periodo	2026-B
Tipo	Formativa (opcional, sin nota)	Entrega	Aula Moodle, según indique el tutor

Objetivos

- Descomponer la versión 1 del reto en componentes según el patrón clásico elegido.
- Declarar las dependencias permitidas y prohibidas entre esos componentes.
- Recorrer un caso de uso con un diagrama de secuencia que use solo esos componentes.
- Escribir reglas de diseño que se puedan verificar en el código.

1. Objetivo

Llegar a la **Actividad de Aprendizaje 2 (20 %)** con el diseño de la versión 1 ya bosquejado y revisado: qué componentes tiene, qué puede usar cada uno, cómo colaboran en el caso de uso más importante y qué reglas no se pueden romper. Esta práctica es **opcional y no calificable**.

Nota: Lo que se produzca aquí es el borrador del diseño UML del **Hito 2** del [proyecto de aula](../../proyecto-aula/). Se trabaja sobre el reto de su equipo y con el patrón clásico que salió del

diagnóstico (Hito 1). Si el diagnóstico aún no está terminado, tome la v1 sugerida en el catálogo para ese reto.

2. Punto de partida

Antes de dibujar, resuma en esta tabla lo que ya decidió el diagnóstico:

Dato	Su reto
Reto y alcance delimitado (H0)	
Atributo de calidad 1 y su medida de respuesta (H1)	
Atributo de calidad 2 y su medida de respuesta (H1)	
Patrón clásico dominante de la v1	
Patrones secundarios, si los hay	
Por qué no los otros (una línea por patrón descartado)	

3. Parte A — Componentes y dependencias permitidas

Dibuje los componentes de la v1 según el patrón elegido:

- **Capas:** una caja por capa, con sus clases o módulos principales dentro.
- **MVC:** controladores, vistas y modelo; si va sobre capas, el modelo se apoya en la capa de negocio.
- **Cliente-servidor:** cada tipo de cliente, el servidor (con su organización interna) y el almacenamiento.
- **Tuberías y filtros:** la fuente, cada filtro, el sumidero y el formato de datos de cada tubería.

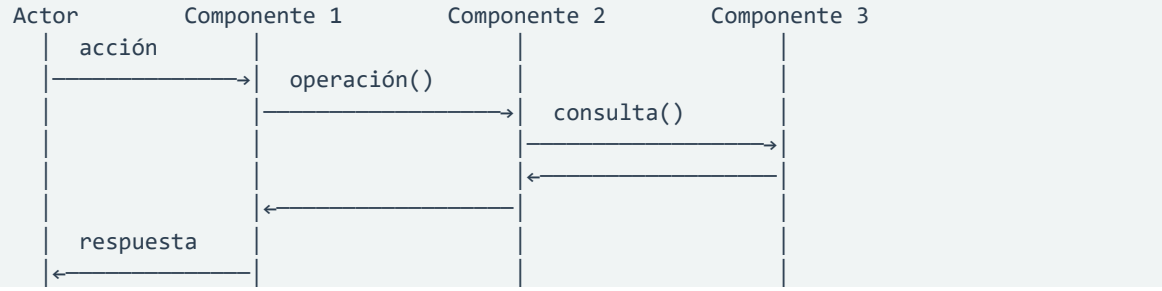
Luego llene una fila por componente. Se piden **al menos cinco** y cada flecha del diagrama debe aparecer en la columna «Puede usar a».

Componente	Responsabilidad (una frase)	Puede usar a	No puede usar a
Ejemplo de Mercado Campesino: PedidoService	Registrar pedidos con los precios vigentes y respetar el cierre del jueves a las 6 p. m.	CatalogoService, PedidoRepository	Controladores, vistas, clases de HTTP

4. Parte B — Un caso de uso de punta a punta

Elija el caso de uso que más pesa en el atributo de calidad dominante del reto. Algunos ejemplos: asignar una cita en CitaSalud, reservar con bloqueo temporal del cupo en RutaSur, procesar una lectura y disparar una alerta en AquaBetania, registrar el ingreso de un lote en TrazaCafé o radicar una petición en PQRS Ciudadana.

Dibuje su diagrama de secuencia con esta forma:



```
camino alterno: qué pasa cuando la regla de negocio rechaza la operación
```

El diagrama cumple cuatro condiciones:

1. Los participantes son **exactamente** componentes de la Parte A.
2. Cada mensaje respeta una dependencia permitida de la Parte A.
3. Se marca **dónde** se aplica la regla de negocio principal (el cupo, el plazo, el umbral).
4. Se incluye **un camino alterno**: cupo agotado, cita ya asignada, lectura inválida, plazo vencido.

Puede hacerse en draw.io, PlantUML, Mermaid o a mano con una fotografía legible.

Tip: Si un mensaje del diagrama no tiene su fila en «Puede usar a», una de las dos partes está mal: o falta la dependencia o sobra el mensaje. Esa misma comprobación cruzada entre diagramas es la que se espera ver en el diseño UML de la Actividad 2.

5. Parte C — Las reglas que el código debe respetar

Escriba **entre cinco y ocho reglas** que el código de la v1 tendrá que cumplir. Una regla útil se puede comprobar: dice qué está prohibido u obligatorio y cómo se verifica.

Regla	Por qué (principio o atributo de calidad)	Cómo se verifica
Ejemplo: La presentación no importa clases de la capa de datos	Separación de responsabilidades; modificabilidad	Revisión de los `import` de los controladores o una prueba de arquitectura
Ejemplo: Ningún filtro lee ni escribe el estado de otro filtro	Independencia de los filtros	Cada filtro tiene pruebas unitarias con solo una entrada y una salida

Las pruebas de arquitectura automatizan este tipo de reglas: en Java, con **ArchUnit**; en Python, con **import-linter**. No son obligatorias en la v1, pero una regla que el código comprueba solo deja de depender de la memoria del equipo.

6. Lista de verificación

- [ ] La tabla del punto de partida nombra el patrón dominante y los dos atributos con su medida.
- [ ] La Parte A tiene al menos cinco componentes, cada uno con una responsabilidad en una frase.
- [ ] Cada componente declara qué puede usar y qué no.
- [ ] Ninguna dependencia contradice el patrón, por ejemplo, de la capa de datos hacia la presentación.
- [ ] El diagrama de secuencia usa solo componentes de la Parte A.
- [ ] Cada mensaje del diagrama corresponde a una dependencia permitida.
- [ ] El diagrama marca dónde se aplica la regla de negocio e incluye un camino alterno.
- [ ] Hay entre cinco y ocho reglas, y cada una dice cómo se verifica.
- [ ] Ningún componente, regla o diagrama es una copia del ejemplo de Mercado Campesino.

7. Entrega

- Un documento **PDF de 2 a 3 páginas** con la tabla del punto de partida, el diagrama y la tabla de componentes, el diagrama de secuencia y la tabla de reglas.
- Nombre sugerido: `U2-practica-<apellido>.pdf`.

- Se sube donde indique el tutor. Al ser práctica opcional, **no tiene nota**: su valor es que el diseño UML de la Actividad 2 salga de algo ya pensado.
- Es **individual**: cada integrante bosqueja la v1 por su cuenta. Comparar los bosquejos dentro del equipo antes de la Actividad 2 permite resolver a tiempo los desacuerdos de diseño.

8. Rúbrica orientativa (para autoevaluación)

Criterio	Logrado	En proceso	Insuficiente
Componentes	Responsabilidades únicas y coherentes con el patrón	Componentes correctos con responsabilidades vagas o repetidas	Una sola caja, o componentes sin relación con el patrón
Dependencias	Permitidas y prohibidas declaradas y coherentes con el patrón	Solo las permitidas	Sin dependencias declaradas, o con ciclos
Caso de uso	Usa los componentes de la Parte A, respeta las dependencias, marca la regla e incluye un camino alterno	Respeta los componentes, pero omite la regla o el camino alterno	Participantes que no existen en la Parte A
Reglas	Cinco a ocho, cada una verificable	Reglas correctas sin forma de verificarlas	Reglas genéricas, como «código limpio»

9. Para ir más allá

- Escriba en el repositorio del equipo una prueba que haga cumplir una de las reglas de la Parte C (ArchUnit en Java, import-linter en Python), de modo que falle si alguien la rompe.
- Anote el escenario de calidad que la v1 **no** resuelve. Ese límite es la pregunta de partida del foro de la Unidad 3 y de la versión 2.